

Tech Stack

TAYLOR LIEGEL, The College of William & Mary, USA

KELLEY WANG, The College of William & Mary, USA

LENA ZHENG, The College of William & Mary, USA

CCS Concepts: • **Human-centered computing** → **Heuristic evaluations**; **User interface programming**; HCI theory, concepts and models; **Heuristic evaluations**; **User interface programming**; HCI theory, concepts and models; • **Information systems** → **Personalization**; *Search interfaces*.

Additional Key Words and Phrases: HCI, Recommending Systems, Technology Frameworks, UI/UX

ACM Reference Format:

Taylor Liegel, Kelley Wang, and Lena Zheng. 2023. Tech Stack. In *CSCI420 Final Project Part F: Report*. ACM, New York, NY, USA, 12 pages.

1 INTRODUCTION

With a fast-paced technology field that is evolving, the barrier to entry is constantly growing for students interested in software development and engineering. Students, and even seasoned professionals, are pressured to know and learn new tools and technologies while still being responsible for existing, popular ones. Because the limits of an application are inherently defined by the technologies chosen to build it, the choice of choosing the right tools for a project is critical. A comprehensive understanding of the available technologies and their trade-offs empowers learners to choose the most suitable tools for their application. When creating a new programming project, it can be daunting and difficult, especially for beginners, to choose the appropriate tools and frameworks to build with.

Software projects often involve multiple frameworks that work with one another, usually in the form of a frontend framework that controls the client-side behavior of the application, a backend framework that controls the server-side and a database system that handles data storage. Typically, graduates and early-career programmers lack exposure to the breadth of options possible for each part and would resort to time consuming and often unreliable online research. In addition to the manual research, if one is lucky enough and finds a good source of information the information may be shown in a way inaccessible to a beginner. Our prototype aims to address these challenges by providing a streamlined process to explore possible tech stack options based on individual project needs, as well as a centralized hub of information to make learning the technologies easier, enabling users to make informed decisions when selecting the ideal tech stack.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

2 BACKGROUND & RELATED WORK

Existing systems are heavily reliant on manual research by the user, as compilations of recommended tools come from person-generated articles and videos. Articles can compare and contrast tools, or list a top number of tools and describe them. Videos do the same function of articles and can also bring the creator's personal experience. Both of these media are reachable through a search engine – which can be seen as the most popular method of initial research in our collected responses. There is variability in the ease of digestibility of information as some content may be less beginner friendly than others.

Two existing tech stack recommenders are found online. One is the development of Tech-Tonic, a tech stack recommender using GPT-3. Upon reviewing the specifics of this project, it was found that Tech-Tonic has a similar end goal in wanting to recommend tech stacks. However, their target audience are developers and project managers, which are both expected to have medium to high levels of experience [8]. Additionally, Tech-Tonic presents an overly simple results page that is difficult to parse for a user with less experience.

Another recent, existing system is StackWise, a tech stack recommender for any ideas using ChatGPT. Much like Tech-Tonic, StackWise seems to be advertised to more experienced persons and “perfect for start-ups and individual developers” [7]. In addition, StackWise's chatbot-like system requires the user to have a clear understanding of what they want for their application and does not take into consideration individual user experiences with different languages and tools – which will increase the difficulty to develop anything for a less experienced user. Beginners need to be provided more details than links to the tool's homepage, especially since some may not know what the terms frontend, backend, or database mean. Our project aims to reach all levels and experiences of software development or engineering, regardless of whether the user is an experienced developer or they are completely new to the technology development field. We will achieve this by asking the users questions regarding their individual tech stack experiences in both breadth and depth. Furthermore, we will present more detailed information about the recommended technologies, such as ratings, basic strengths and weaknesses, and installation help.

Our product would target a specialized demographic of users who are interested in application development with some baseline level of skill with using technology and programming languages, whether through formal education or independent research. We also require the input of users, regarding familiarity with select technologies, tools, or languages, to generate the most helpful tool recommendations for our users.

In this paper, we investigate the current need for a tech stack recommender application that can provide a suitable starting point for anyone who is interested in development. We adopted a co-design process that involves stakeholders participating in the design process, providing feedback and identifying problems. We surveyed 11 undergraduate computer science students who would be directly benefited by the development of our tech stack recommender. We also presented preliminary design ideas to an Intro to Human Computer Interaction (HCI) class of 39 people and received feedback on the design of this project and presentation of the tool information. In another session, we received feedback on our final design idea, which we utilized to make final design edits.

After considering abundant comments and reviewing design ideas, we designed a tech stack recommender system that is suitable for people of all experience levels. This design incorporates concepts of HCI such as user experience and user interaction (UI/UX) and is accessible to everyone. This project can serve to guide new developments for future tech stack generation technology.

3 DESIGN

Our design process consisted of multiple iterations over a planned prototype targeted towards an imagined user persona. The imagined persona [fig. 2] was that of an inexperienced Computer Science student or recent graduate that had a desire to build personal or professional projects, but lacked the technical knowledge to know what tech stack to choose to start building. The individuals in this group acted as our primary shareholders who would be most affected by our project. We also created a storyboard [fig. 3] to visualize the problem space and how a member of this persona group could utilize our product for a better experience in choosing a tech stack.

Prior to beginning our design, we first sought to gather information about this demographic to better suit our product to their specific needs. We did this through a qualtrics survey that had the goals of discovering the current solutions users employed in the problem space, their frustrations with these solutions, and relevant information like their graduation year and programming strengths and weaknesses. This qualtrics survey was given to either graduates in Computer Science, or those who currently were majoring in or had an interest in it [11]. The survey had 11 participants.

Building off of the information obtained from our research, we created two low-fidelity prototypes [fig. 4,5] of our project demonstrating potential design and functionality. These prototypes together suggested a survey like user experience that acted as a recommending system to suggest technical frameworks in the form of “card stacks”. They also allowed for users to obtain details about the individual technologies (technical and personal) in a compartmentalized window format that could be closed and opened to minimize visual clutter. In the next step in our design process, we presented our prototypes to the class and received feedback to iterate further. Some of the ideas mentioned in the feedback were to reduce the busy redesign of the prototypes as well as leverage existing design paradigms to emphasize recognition over recall to improve the user experience.

Based on this feedback, we reiterated, and produced our first high-fidelity prototype. We focused on creating a minimalist UI that used a simple green color palette to reduce the design complexity but still remain accessible to those with color blindness. To emphasize user control and knowledge of the state of the survey, we added more navigation tools, and a progress bar that informed users how far they were in the survey for each step of the process.

For the next step, we performed a user evaluation on this prototype, the details of which are discussed in section four. Within this process, we utilized Jakob Nielsen’s ten general principles for interaction design [10]. Based on this feedback, we decided to alter the existing high fidelity prototype to improve the user experience. These alterations included a way to go back to the survey process and edit results following completion, functionality that checked with the user to confirm they were ready to submit, changes to the ranking functionality that reduced required user effort, and altering the existing UI elements like buttons to be responsive to interaction.

We presented this new prototype again to the class, and received new feedback. This feedback emphasized demonstration of the nature between frameworks, separation between industry standard recommendations versus beginner friendly ones, and a color coding system to indicate difficulty.

For the final prototype, we incorporated this feedback to make specific changes. One change we made was to restructure the “Results” page such that two different stacks are offered as a result. One stack is beginner-oriented and easier to use, while the other is more challenging but the standard within the industry. This page offers users the choice between two options to facilitate more use cases. If a user is creating a project with the goal of getting a product created as fast as possible, the easier framework may be the best choice for them. However, the same may not be true for a user who is seeking to build a project specifically with the purpose to learn new technologies and expand their skill sets. We

also added regeneration buttons on each card that allow the user to switch out the technology framework that fulfills a specific role with another one that fits, as well as help tooltips the user can toggle on the results page.

The link to our final prototype can be found [here](#).

4 EVALUATION

Our evaluation was a combination of informal and formal techniques. After our two low-fidelity prototypes were created, we pitched them to our peers, who happened to also be part of our primary stakeholders. Many of their intuitive suggestions fell into categories of Nielsen's 10 Design Principles, and these inspired development on a better prototype.

After our first iteration of our high-fidelity prototype, we performed usability tests by interviewing two participants in a controlled setting. Both users were Computer Science students so they were applicable as part of our primary stakeholder demographic. The main form of documentation of their reviews was the heuristic evaluation usability bug report, which more formally incorporates Nielsen's ten general principles for interaction design. For each violation found, the user described it, what heuristic it specifically violates, and its severity. The results of our testers show that the principles violated with the highest scores in severity were "user control & freedom," "help & documentation," and "visibility of system status" with scores of 3, 4, and 3 respectively. This ranking system helped us inform what bugs in our initial high-fidelity prototype would be best prioritized to be worked on.

Notably, our tests found our prototype had violations in every design principle except for "aesthetic & minimalist design." This suggests our prototype is strong in communicating essential information to users, and further refinement of this aspect, although nice, would not necessarily be vital in producing a compelling interface.

We presented our second iteration to our peers, who at this point were fully informed of the ten design principles for interaction design. We were able to receive feedback, once again in an informal setting, and produced the final version of our high-fidelity prototype.

5 RESULTS

Within our user evaluations, testers found several limitations within our prototype that provided an opportunity for us to iterate further and improve the design. The limitations with the highest severity dealt with the ability of the user to control the process, the amount of contextual information provided and the responsiveness of the state of the prototype in response to user action. User control was negatively impacted by insufficient navigation tools within the survey process that also utilized symbols that relied on the user to recall the meaning of the symbols rather than being able to immediately recognize their purpose. Similarly, in the original prototype, there was no way for users to go back and modify responses after the results were shown, forcing them to re-do the entire process if they realized the mistake after the fact. Furthermore, buttons did not change after user interaction, resulting in subpar system responsiveness to state changes. Another high priority problem was that test users evaluating the product found the contextual information provided in the survey to be lacking, and potentially inaccessible to users unfamiliar with specific jargon.

These failures found during the user evaluations could be realistically remedied, and we were able to successfully modify the prototype design to reduce their impact on the user experience. Other successes included that the users found no problem with the existing aesthetics of the design and color scheme, and through objective testing we demonstrated that it was accessible for those with colorblindness [fig. 3].

However, during the evaluation process and the feedback obtained from the in-class presentation, there were some limitations that still persisted after our initial fixes. One piece of feedback was that our product lacked in the heuristic of error prevention and response in the case that no viable stack was found that fit the users response. Similarly,

another suggestion was to provide additional stack options on the results page in case the user found the main results to be unsatisfactory for their needs. Furthermore, another individual who had seen our product suggested that the context provided by the results page was insufficient, and to give a better insight into the interplay between different frameworks, a critical part of the development process, we should include a visual, potentially in the form of a graph, to display this information. While we failed to adequately address these problems in our initial iteration of the prototype, we aim to resolve them through re-iteration if possible to improve the user experience.

6 DISCUSSION

A recommender system, or RS, is a tool that filters a large amount of information using user preferences to return personalized advice or options [1]. These systems are widely used in online commercial spaces and networking platforms because they are effective in providing users with satisfactory results that enable further interaction [2].

For a significant period of time, research pertaining to RS primarily focused on optimizing algorithms based on general metrics, like MSE, rather than optimizing based on RS-user interaction [3]. Following significant expansion of the field because of commercial incentives, such as the Netflix Prize which offered one million dollars to anyone who could improve Netflix's RS, focus on RS-user interaction grew and began to incorporate HCI principles [3,4]. Since then, HCI has become a valuable aspect in the creation of RSes as a "growing consensus" formed that the "effectiveness of [research systems] goes beyond recommendation accuracy [5 p.124]" and that "small changes" to accuracy "were not the path to significant improvements in user experience [3 p.105]". Within the realm of HCI, recent research focuses on modifying existing structures of the standard research system interface, whether that be through increased user input and analysis of response as shown in Calero et al., or through conversational methods like in Lin et al.

Our product is a recommender system tailored towards the specific needs of creating a technology stack. It is designed to leverage the standardized inputs of the user to filter a large body of information into easy to understand results. In the context of HCI, our product aligns with the recently popular view that a recommender system cannot thrive alone on algorithmic accuracy alone, but needs to incorporate user-oriented data input and processes as well. We have done this through incorporating user feedback from class presentations and user evaluations, and by trying to adhere to the design heuristics.

Few existing solutions exist specifically within our problem space pertaining to tech stack generation. One example is StackWise which is a barebones ChatGPT chatbot which relies on users knowing exactly what inputs are relevant to provide to the service, and lacks the design interface our product emphasizes [7]. Another example is a Hackathon project Tech Tonic that provides a user interface to provide user preferences, but violates several design heuristics and would be difficult to parse for an inexperienced user not familiar with technical jargon [8]. For example, the output screen contains large text blocks that make it challenging for users to separate each component, and uses specialized jargon like "MERN stack" to deliver results.

The strengths of our project are that it improves on the user interfaces of existing implementations within the problem space and aims to provide accessibility to all skill levels. In its current state, some weaknesses are the limited amount of information it displays about the chosen results, specifically very little in regards to how the interconnectivity between the different frameworks functions. Additionally, the current format of the survey is relatively rigid, through allowing users to weight different priorities differently, we could allow significantly more flexibility within the process for discovery. We could also attempt to overcome this rigidity by employing alternative solutions that pull from existing research, such as through conversational recommender systems or utilization of collaborative filtering, a type of RS that incorporates information from other users [6,9].

7 CONCLUSION

Overall, the key findings were that this project would help fill a void for solutions within the current problem space. This is significant, because although many people may struggle with choosing a tech stack for their projects, due to the lack of available options, they often result to frustrating, time-intensive manual effort. There are few existing solutions that not only perform the task, but also provide results in an accessible and easily understood format that aligns with HCI standards and user-focused design.

Future efforts should focus on expanding the functionality of the current prototype, specifically in regards to accounting for the potential complexity of connecting the different frameworks, and also in the way the users interact with the framework. Interactive technology, such as a chat-bot interface could be beneficial to users to tailor their results. The survey questions themselves could also be more robust to account for more information about the user.

To conclude, the project is a part of the continuing trend of user-focused recommending systems by prioritizing user focused design. If fully implemented, the project could potentially be helpful to those interested in computer science, and help spur more innovation by helping to streamline the creation process.

8 FIGURES

Color-blindness Comparison

Error message is still visible.

Part 2: Experience How many years of experience do you have in software development? ! Not a valid number. Please enter a valid number to continue. Enter a number > 0	Part 2: Experience How many years of experience do you have in software development? ! Not a valid number. Please enter a valid number to continue. Enter a number > 0	Part 2: Experience How many years of experience do you have in software development? ! Not a valid number. Please enter a valid number to continue. Enter a number > 0	Part 2: Experience How many years of experience do you have in software development? ! Not a valid number. Please enter a valid number to continue. Enter a number > 0
No color blindness	Protanopia (red-blind)	Deuteranopia (green-blind)	Tritanopia (blue-blind)

Part 2: Experience How many years of experience do you have in software development? ! Not a valid number. Please enter a valid number to continue. Enter a number > 0	Part 2: Experience How many years of experience do you have in software development? ! Not a valid number. Please enter a valid number to continue. Enter a number > 0
Blue cone monochromacy	Monochromacy/Achromatopsia

Simulated images sourced from:

<https://pilestone.com/pages/color-blindness-simulator-1#>

Fig. 1. Results of color blindness test for error screen from pilestone.com/pages/color-blindness-simulator.

	Bio A Senior level / New grad C.S student with a desire to build a piece of software but doesn't know where to start. Has a technical background but lacks expert level knowledge.	Motivations: • Get basis for tech stack that is ① Fit for his product ② Realistic to learn
Name: Tim CStudent Age: 21 Work: College Student in C.S Family: Unmarried, no kids. Income: < 10,000 / yr	Pain Points: • Not sure where to start: getting lost in the clutter of avail. resources. • Lack of experience to make decisions.	Goals: ① Gain experience in new technology ② Make a product that is at least something to talk about in an interview or that can be used on a large scale.

Fig. 2. Our product user persona

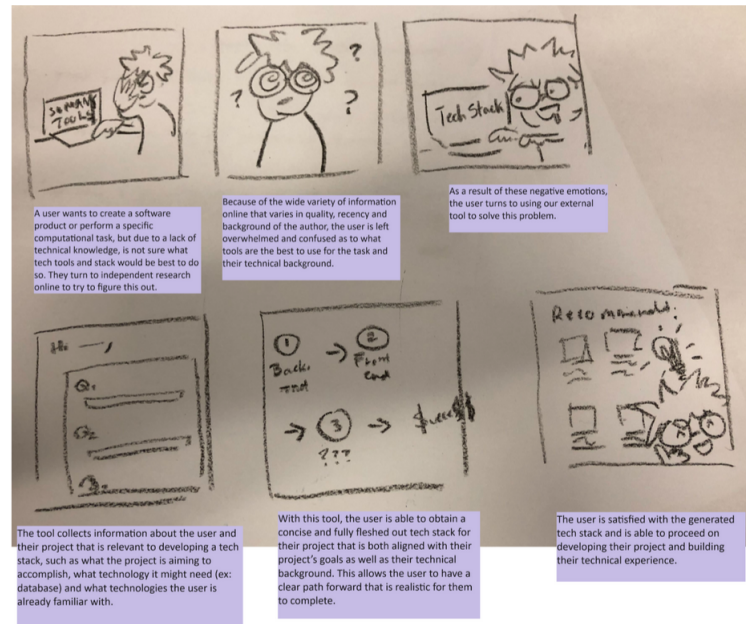


Fig. 3. Our product storyboard

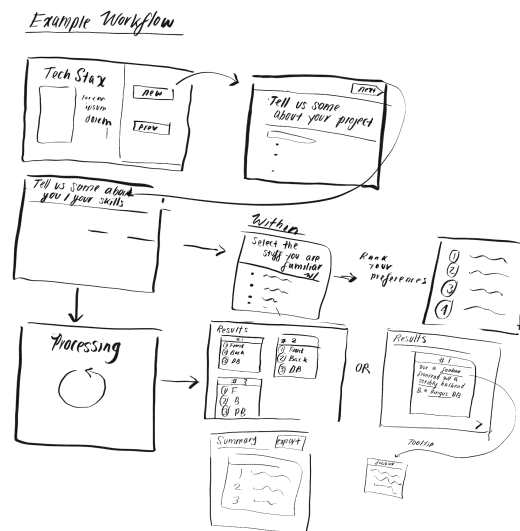


Fig. 4. Our first low fidelity prototype



Fig. 5. Our second low fidelity prototype

9 CITATIONS AND BIBLIOGRAPHIES

ACKNOWLEDGMENTS

Special thanks to Professor Yixuan "Janice" Zhang who taught our Introduction to Human-Computer Interaction class.

REFERENCES

- (1) Christos Troussas, Akrivi Krouska, and Cleo Sgouropoulou. 2021. Enhancing human-computer interaction in digital repositories through a MCDA-based Recommender System, *Advances in Human-Computer Interaction* 2021, pp. 1–3, 2021. <https://doi.org/10.1155/2021/7213246>
- (2) Umberto Panniello, 2014. How to Use Recommender Systems in e-Business Domains, *Webology* 11, pp.2-4, 2014. Retrieved December 18, 2023 from <https://www.webology.org/abstract.php?id=184>
- (3) Joseph A. Konstan and John Riedl. 2012. Recommender systems: From algorithms to user experience. *User Modeling and User-Adapted Interaction* 22, 1-2, 101-123. Retrieved December 18, 2023 from <https://doi.org/10.1007/s11257-011-9112-x>.
- (4) Joseph A. Konstan and Loren G. Terveen. 2021. Human-centered recommender systems: origins, advance, challenges and opportunities. *AI Magazine* 42, 3, 31-42. Retrieved December 18, 2023 from <https://doi.org/10.1609/aimag.v42i3.18142>.
- (5) André Calero Valdez, Martina Ziefle and Katrien Verbert. 2016. HCI for Recommender Systems: the Past, the Present and the Future. *RecSys '16: Proceedings of the 10th ACM Conference on Recommender Systems* (Sep. 2016) 123-126. Retrieved December 18, 2023 from <https://doi.org/10.1145/2959100.2959158>
- (6) Allen Lin, Ziwei Zhu, Jianlin Wang, and James Caverlee. 2023. Enhancing user personalization in conversational recommenders. *ACM Digital Library*. <https://doi.org/10.1145/3543507.3583192>
- (7) div3rsity. 2023. StackWise - The Teck Stack finder for any Idea. *FlowGPT* Retrieved December 17, 2023 from <https://flowgpt.com/p/tech-stack-finder>.
- (8) Puja Agarwal. 2023. Tech-Tonic. *Devpost*. Retrieved December 17, 2023 from <https://devpost.com/software/tech-tonic>.
- (9) Yehuda Koren and Robert Bell. 2015. Advances in collaborative filtering, *Recommender Systems Handbook*, pp. 77–118, 2015. <https://doi.org/10.1007/978-1-4899-7637-6>
- (10) Jakob Nielsen. 1994. 10 usability heuristics for user interface design. (April 1994). Retrieved December 18, 2023 from <https://www.nngroup.com/articles/ten-usability-heuristics/>

(11) https://wmsas.qualtrics.com/jfe/form/SV_3KsnRxPnosQqeI6

APPENDIX

A HEURISTIC EVALUATION USABILITY BUG REPORT

A.1 Bug Report 1

Individual Bug Reports:

User 1 Bug Report (Melody Linas):

Heuristic Evaluation Usability Bug Report

Prototype: Tech Stack

Evaluator: Melody Linas

Heuristic(s) violated: Recognition Rather than Recall

Brief description of the issue raised:

- The navigation buttons are far away from each other and quite small on the screen so they are not eye catching. Additionally because they are so far away a user needs to move their mouse across a significant portion of the screen in order to navigate.
- Suggestion: Make navigation buttons bigger, and at bottom right corner, also next to each other but not too close.
- Consider maybe making these buttons have words "next" or "back" since your UI looks pretty minimal, adding the words give the buttons more weight on the screen therefore easier to see.

Severity: 1

(0 to 4, where 4 is most severe, based on frequency of occurrence, impact on user experience, persistence of issue with increased use of prototype, and market impact)

----- Heuristic Evaluation Usability Bug Report

Heuristic(s) violated: Match between system and real world

Brief description of the issue raised:

- Assuming target users are mostly beginners in using tech stacks, instead of saying "full stack" "front end" "back end" maybe include either examples of what exactly each of those entails (ex: website → front end focused). If I am a complete beginner I might not be entirely sure of the scope of my project yet or what classifies under what.

Severity: 2

(0 to 4, where 4 is most severe, based on frequency of occurrence, impact on user experience, persistence of issue with increased use of prototype, and market impact)

----- Heuristic Evaluation Usability Bug Report

Heuristic(s) violated: User Control & Freedom

Brief description of the issue raised:

- Back button disappears when landing on the ending page, this could be very inconvenient if the user accidentally clicks to the last page and wants to change something but they can't, causing them to start over.

Severity: 3

(0 to 4, where 4 is most severe, based on frequency of occurrence, impact on user experience, persistence of issue with increased use of prototype, and market impact)

Heuristic Evaluation Usability Bug Report

Heuristic(s) violated: Error Prevention

Brief description of the issue raised:

- Currently it just immediately takes the user to the final page without any feedback from them first.
- Maybe include a submit button and asking the user "Are you sure you're ready to submit?" otherwise they could accidentally click next

Severity: 2

(0 to 4, where 4 is most severe, based on frequency of occurrence, impact on user experience, persistence of issue with increased use of prototype, and market impact)

Heuristic Evaluation Usability Bug Report

Heuristic(s) violated: Flexibility and efficiency of use

Brief description of the issue raised:

- Doesn't really allow users to skip survey portion, maybe allow them a button to just skip to view the tech stack info/documentation up front

Severity: 1

(0 to 4, where 4 is most severe, based on frequency of occurrence, impact on user experience, persistence of issue with increased use of prototype, and market impact)

Heuristic Evaluation Usability Bug Report

Heuristic(s) violated: Help users recognize, diagnose, and recover from errors

Brief description of the issue raised:

- Platform does not inform users (if there is an error) what is going on.
- Suggestion: Maybe have a page where let's say, the user's project preference is too niche, i.e. no tech stack really fits the project, then have a page with "We couldn't find a stack that fits your description, would you like to adjust your search?" — this "offers a solution"

Severity: 1

(0 to 4, where 4 is most severe, based on frequency of occurrence, impact on user experience, persistence of issue with increased use of prototype, and market impact)

Heuristic Evaluation Usability Bug Report

Heuristic(s) violated: Help & Documentation

Brief description of the issue raised:

- Does not provide documentation on how to install said tech stacks
- Could include links to videos that give an intro to said tech stack

Severity: 3

(0 to 4, where 4 is most severe, based on frequency of occurrence, impact on user experience, persistence of issue with increased use of prototype, and market impact)

A.2 Bug Report 2

User 2 Bug Report (Kelley Wang):

Heuristic Evaluation Usability Bug Report

Prototype: Tech Stack
Evaluator: Kelley Wang

Heuristic(s) violated: Visibility of System Status

Brief description of the issue raised:

- In Part 1, there is no visible change on the screen when I select a single choice. The visual cues in the current prototype do not give me information to know if my desired choice has been selected.

Severity: 3

(0 to 4, where 4 is most severe, based on frequency of occurrence, impact on user experience, persistence of issue with increased use of prototype, and market impact)

----- Heuristic(s) violated: Match between the system and real world

Brief description of the issue raised:

- The user may not be familiar with the terms "full-stack," "front-end," or "back-end."
- If a user doesn't know any of these terms they might feel forced to choose "Other."

Severity: 2

(0 to 4, where 4 is most severe, based on frequency of occurrence, impact on user experience, persistence of issue with increased use of prototype, and market impact)

----- Heuristic(s) violated: User control & freedom

Brief description of the issue raised:

- Once I reach the result page I am unable to go back and change survey answers. The previous page from results leads to a loading animation which redirects the user to the same results.
- Although the home button offers an escape, if users want different results they must re-answer the whole survey.

Severity: 2

(0 to 4, where 4 is most severe, based on frequency of occurrence, impact on user experience, persistence of issue with increased use of prototype, and market impact)

----- Heuristic(s) violated: Consistency and standards

Brief description of the issue raised:

- In the results, clicking on the left card results in an animated transition to a page with more details. However, the middle and right cards lack this animation. There is an internal

inconsistency.

Severity: 1

(0 to 4, where 4 is most severe, based on frequency of occurrence, impact on user experience, persistence of issue with increased use of prototype, and market impact)

----- Heuristic(s) violated: Consistency and standards

Brief description of the issue raised:

- Links look like they would be clickable. I think most users expect a link would redirect them to a page.
- Other applications allow clickable links. External consistency with these outside products would meet a user expectation.

Severity: 2

(0 to 4, where 4 is most severe, based on frequency of occurrence, impact on user experience, persistence of issue with increased use of prototype, and market impact)

----- Heuristic Evaluation Usability Bug Report

Heuristic(s) violated: Help and documentation

Brief description of the issue raised:

- There is neither a help/document page nor a way to reach to directly reach out to Tech Stack.
- There is no help or documentation page. The user may want to directly reach out to Tech Stack for questions.

Severity: 2

(0 to 4, where 4 is most severe, based on frequency of occurrence, impact on user experience, persistence of issue with increased use of prototype, and market impact)

B LINK TO FINAL PROTOTYPE(S)

Wireframe/Developer View
Interactive Prototype View